UNIVERSIDAD TECNOLÓGICA DE PEREIRA
FACULTAD DE INGENIERÍAS
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
PROGRAMACIÓN III

Actividad 05 – Estructuras y listas

Integrante: Esteban Guevara Cardona

Enlace a los soportes: https://github.com/EstebanGC222/Actividad03-Casos-con-Listas

1. Árbol genealógico (familia Simpson)

Los hombres y las mujeres quedan en dos listas. Los hijos de cada progenitor también están en una lista, y el progenitor se representa con la estructura padre(Nombre) o madre(Nombre). Las reglas padre/2, madre/2, hombre/1 y mujer/1 ahora buscan dentro de esas listas, y hermanos/2 revisa si dos personas están en la misma lista de hijos.

Programa

```
% Hechos: listas de hombres y mujeres
hombres([abraham, clancy, herbert, homero, bart]).
mujeres([mona, jacqueline, marge, patty, selma, lisa, maggie, ling]).

% Hechos: hijos(Progenitor, ListaDeHijos)
% Progenitor es una estructura: padre(Nombre) o madre(Nombre)
hijos(padre(abraham), [herbert, homero]).
hijos(padre(clancy), [marge, patty, selma]).
hijos(padre(homero), [bart, lisa, maggie]).
hijos(madre(mona), [homero]).
hijos(madre(jacqueline), [marge, patty, selma]).
hijos(madre(marge), [bart, lisa, maggie]).
hijos(madre(selma), [ling]).

% Pertenencia a una lista (cabeza y cola)
pertenece(X, [X|_]).
pertenece(X, [_|Cola]) :- pertenece(X, Cola).

% Reglas
hombre(X) :- hombres(L), pertenece(X, L).
mujer(X) :- mujeres(L), pertenece(X, L).

padre(X, Y) :- hijos(padre(X), L), pertenece(Y, L).
madre(X, Y) :- hijos(madre(X), L), pertenece(Y, L).

progenitor(X, Y) :- padre(X, Y).
progenitor(X, Y) :- madre(X, Y).

% Hermanos: están en la misma lista de hijos
hermanos(X, Y) :- hijos(padre(_), L), pertenece(X, L), pertenece(Y, L), X \= Y.
hermanos(X, Y) :- hijos(madre(_), L), pertenece(X, L), pertenece(Y, L), X \= Y,
                  \+ (padre(P, X), padre(P, Y)).

abuelo(X, Y) :- padre(X, Z), progenitor(Z, Y).
abuela(X, Y) :- madre(X, Z), progenitor(Z, Y).

nieto(X, Y) :- hombre(X), (abuelo(Y, X) ; abuela(Y, X)).
```

```
nieta(X, Y) :- mujer(X), (abuelo(Y, X) ; abuela(Y, X)).

tio(X, Y) :- hombre(X), progenitor(Z, Y), hermanos(X, Z).
tia(X, Y) :- mujer(X), progenitor(Z, Y), hermanos(X, Z).

primos(X, Y) :- progenitor(A, X), progenitor(B, Y), hermanos(A, B).
```

Consultas

```
?- abuelo(X, bart).
X = abraham ;
X = clancy.

?- abuela(X, ling).
X = jacqueline.

?- nieta(X, jacqueline).
X = lisa ;
X = maggie ;
X = ling.

?- hermanos(herbert, X).
X = homero.

?- tio(X, lisa).
X = herbert.

?- tia(X, bart).
X = patty ;
X = selma.

?- primos(bart, X).
X = ling.

?- abuelo(homero, bart).
false.
```

2. Grafo dirigido (ciudades de Canadá)

Cada ciudad tiene una lista con sus aristas de salida, y cada arista es la estructura arista(Destino, Costo). La regla arista/3 busca dentro de esa lista, tiene_aristas/1 revisa si la lista tiene al menos un elemento ([_|_]), y viajar/4 usa una lista de ciudades visitadas para no repetir nodos.

Programa

```
% Hechos: ciudad(Nombre, ListaDeAristas)
% Cada arista es una estructura: arista(Destino, Costo)  -- grafo dirigido
ciudad(vancouver, [arista(edmonton, 16), arista(calgary, 13)]).
ciudad(edmonton, [arista(saskatoon, 12)]).
ciudad(calgary, [arista(edmonton, 4), arista(regina, 14)]).
ciudad(saskatoon, [arista(calgary, 9), arista(winnipeg, 20)]).
ciudad(regina, [arista(saskatoon, 7), arista(winnipeg, 4)]).
ciudad(winnipeg, []).

% Pertenencia a una lista (cabeza y cola)
```

```
pertenece(X, [X|_]).
pertenece(X, [_|Cola]) :- pertenece(X, Cola).

% Arista de X a Y con costo C: está en la lista de aristas de X
arista(X, Y, C) :- ciudad(X, L), pertenece(arista(Y, C), L).

% Conexión directa entre dos nodos, en cualquier sentido
conexion(X, Y, C) :- arista(X, Y, C).
conexion(X, Y, C) :- arista(Y, X, C).

% Un nodo tiene aristas si su lista no está vacía o si le llega alguna
tiene_aristas(X) :- ciudad(X, [_|_]), !.
tiene_aristas(X) :- arista(_, X, _), !.

% Costo de ir de X a Z pasando por Y
costo(X, Y, Z, C) :-
    arista(X, Y, C1),
    arista(Y, Z, C2),
    C is C1 + C2.

% ¿Es posible viajar de X a Y? La lista Visitados evita repetir nodos
viajar(X, Y, Ruta, Costo) :- viajar(X, Y, [X], R, Costo), reverse(R, Ruta).
viajar(X, X, Visitados, Visitados, 0).
viajar(X, Y, Visitados, Ruta, Costo) :-
    arista(X, Z, C),
    \+ pertenece(Z, Visitados),
    viajar(Z, Y, [Z|Visitados], Ruta, C2),
    Costo is C + C2.
```

Consultas

¿Existe una conexión entre Saskatoon y Vancouver?

```
?- conexion(saskatoon, vancouver, C).
false.

?- viajar(saskatoon, vancouver, Ruta, Costo).
false.
```

¿Con qué nodos está conectado Regina y cuál es el costo de cada conexión?

```
?- conexion(regina, X, Costo).
X = saskatoon, Costo = 7 ;
X = winnipeg, Costo = 4 ;
X = calgary, Costo = 14.
```

Regla para determinar si un nodo tiene aristas:

```
?- tiene_aristas(regina).
true.

?- tiene_aristas(bogota).
false.
```

Regla para el costo de ir de un nodo X a un Z pasando por Y:

```
?- costo(vancouver, calgary, regina, C).
C = 27.

?- costo(vancouver, Y, saskatoon, C).
```

```
Y = edmonton, C = 28.
```

¿Es posible viajar desde Edmonton a Calgary?

```
?- viajar(edmonton, calgary, Ruta, Costo).
Ruta = [edmonton, saskatoon, calgary], Costo = 21.
```